

# Assignment 1

## Part A: Conceptual Questions

### 1. What is Full Stack Development?

Full Stack Development is the process of developing both the frontend (user interface) and backend (server-side logic) of a web application.

A Full Stack Developer works on:

- Frontend (HTML, CSS, JavaScript)
- Backend (Java, Node.js, Python, PHP, etc.)
- Database (MySQL, MongoDB, etc.)

**Example:** An Amazon developer who creates the website interface, processes orders, and stores customer data is doing full stack development.

### 2. What is Frontend?

Frontend is the part of a website that users can see and interact with directly.

**Technologies used in Frontend:**

- HTML
- CSS
- JavaScript
- React.js
- Bootstrap

**Example:** Login page, buttons, images, forms, and menus.

### 3. What is Backend?

Backend is the server-side part of a website that processes requests and handles business logic.

**Main Responsibilities:**

- Process user requests
- Connect with databases

- Validate data
- Manage authentication
- Send responses to users

**Example:** Checking if a username and password are correct during login.

## 4. What is a Database?

A database is a collection of organized data that can be stored, managed, and retrieved easily.

Websites need databases to store information permanently.

**Examples of data stored in databases:**

- User accounts
- Passwords
- Product details
- Orders
- Student records

**Database Examples:**

- MySQL
- MongoDB
- PostgreSQL

## 5. What is DNS?

DNS (Domain Name System) converts a website name into an IP address.

When a user enters a website URL, DNS helps find the correct server where the website is stored.

**Example:**

www.google.com → 142.250.xxx.xxx

Without DNS, users would need to remember IP addresses instead of website names.

## 6. What is an IP Address?

An IP Address is a unique number assigned to every device connected to a network.

**Example:**

192.168.1.1

**Importance in Networking:**

- Identifies devices
- Helps communication between devices
- Allows data to reach the correct destination

## **7. What is Client-Server Architecture?**

Client-Server Architecture is a model where a client requests services and a server provides them.

**Communication Process:**

1. User sends a request from the browser.
2. Server receives the request.
3. Server processes the request.
4. Server sends a response back.
5. Browser displays the result.

**Example:** Opening YouTube in a browser.

## **8. What is an API?**

API (Application Programming Interface) allows two applications to communicate with each other.

**Role of APIs in Web Applications:**

- Exchange data between frontend and backend
- Connect third-party services
- Fetch or send information

**Example:** Weather API provides weather information to a weather app.

## **9. What are HTTP Methods?**

HTTP Methods define the action to be performed on data.

### **GET**

Used to retrieve data from a server.

**Example:** Fetching product details.

### **POST**

Used to send new data to the server.

**Example:** Submitting a registration form.

### **PUT**

Used to update existing data.

**Example:** Updating a user profile.

### **DELETE**

Used to remove data.

**Example:** Deleting a product from a cart.

## **10. What are Developer Tools?**

Developer Tools are browser tools used to inspect, debug, and analyze websites.

### **Common Tabs and Their Purpose:**

#### **Elements Tab**

Used to inspect HTML and CSS.

#### **Network Tab**

Shows requests sent between browser and server.

#### **Console Tab**

Displays errors, logs, and JavaScript output.

# Part B: Diagram Activities

## 11. Full Stack Architecture Diagram

```
User
↓
Frontend
↓
Backend
↓
Database
```

### Description

#### User

- Person using the website.

#### Frontend

- Displays information and takes user input.

#### Backend

- Processes requests and business logic.

#### Database

- Stores and manages application data.

## 12. Complete Website Working Flow

```
User
↓
Browser
↓
DNS
↓
Server
↓
Database
↓
Server
↓
```

Browser  
↓  
Website Displayed

## **Explanation**

### **User**

- Enters a website URL.

### **Browser**

- Sends a request to access the website.

### **DNS**

- Converts the domain name into an IP address.

### **Server**

- Receives and processes the request.

### **Database**

- Provides required data to the server.

### **Server**

- Generates a response.

### **Browser**

- Receives the response and renders the webpage.

### **Website Displayed**

- User can now view and interact with the website.

## **Part C: Real-World Application Questions**

### **13. Student Registration System**

A student fills a registration form on a website.

### **What happens in the Frontend?**

- Student enters details.
- Form validates basic information.
- Data is sent to the backend.

### **What happens in the Backend?**

- Receives registration data.
- Checks whether data is valid.
- Processes the registration request.

### **What happens in the Database?**

- Stores student information.
- Saves records permanently.
- Makes data available when needed.

## **14. Amazon Search Scenario**

A user searches for "Laptop" on Amazon.

1. The user types "Laptop" in the browser.
2. The browser sends a request to the server.
3. The server searches the database.
4. The database returns matching laptop records.
5. The server prepares a response.
6. The response is sent back to the browser.
7. The browser displays the search results.

**Terms Used:** Browser, Server, Database, Response.

## **15. Website Observation Activity**

### **Website Name**

Google

### **Observation from Elements Tab**

The webpage contains HTML elements such as buttons, input fields, and images.

## **Observation from Network Tab**

Multiple requests are sent to the server when the page loads.

## **Observation from Console Tab**

JavaScript messages and website logs are displayed without errors.

## **16. Reflection Question**

### **How does a website work from URL entry to webpage display?**

When a user enters a URL in a browser, the browser contacts DNS to find the website's IP address. The request is then sent to the server. The server processes the request and retrieves data from the database if required. After processing, the server sends a response back to the browser. Finally, the browser renders the HTML, CSS, and JavaScript files and displays the webpage to the user.

## **17. Difference Between Frontend, Backend, and Database (Amazon Example)**

### **Frontend**

The frontend is what customers see on Amazon, such as product pages, search bars, images, and buttons.

### **Backend**

The backend processes user actions, handles orders, verifies payments, and communicates with the database.

### **Database**

The database stores product information, customer details, order history, and inventory records.

### **Example Flow**

When a customer searches for a laptop:

- Frontend takes the search input.
- Backend processes the search request.
- Database returns matching products.



- Backend sends results back.
- Frontend displays the products on the screen.

This shows how frontend, backend, and database work together in a real-world application like Amazon